Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

(Университет ИТМО)

Факультет **Прикладной информатики**

Образовательная программа **Автоматизация разработки и управления информационными системами**

Направление подготовки(специальность) **Инфокоммуникационные технологии и системы связи**

ПРАКТИЧЕСКАЯ РАБОТА № 1

По дисциплине «Основы программирования»

Тема: Выполнение программы, объекты и базовые типы Python

Выполнил **Михеев И. А.**

Проверил

Дата _____

Санкт-Петербург 2026

ЗАДАНИЕ 1

Содержимое файла task_1.py выглядит следующим образом:

```
print(2 + 3 * 4)

course = "Python"
hours = 4 * 2
print(f"{course}: {hours} часов")
```

К выражениям в данном файле относится:

1) Вычисление арифметического выражения 2 + 3 * 4 в строке 1.

2) Вычисление арифметического выражения 4 * 2 в строке 4.

3) Конкатенация f-строки f"{course}: {hours} часов" в строке 5.

К инструкциям в данном файле относится:

1) Присваивание выражения к переменной course в строке 3.

2) Присваивание выражения к переменной hours в строке 4.

3) Вызов функции print() в строке 5.

Какие литералы присутствуют:

– Числовые 2, 3, 4, 4, 2

– Строковые "Python", f"{course}: {hours} часов"

Список имён и их назначение:

– print — имя встроенной функции Python, которая выводит текст или результаты на экран.

– course — имя переменной, куда сохраняется название курса.

– Python — имя (или значение без кавычек), которое присваивается переменной course.

– hours — имя переменной, куда сохраняется результат математического вычисления.

AST и DIS код

Моя версия интерпретатора: Python 3.14.3.

AST код присваивания:

```
Assign(
         targets=[
            Name(id='course', ctx=Store())],
         value=Constant(value='Python')),
```

```
      Assign(
         targets=[
            Name(id='hours', ctx=Store())],
         value=BinOp(
            left=Constant(value=4),
            op=Mult(),
            right=Constant(value=2))),
```

AST код арифметических операций:

```
BinOp(
                  left=Constant(value=2),
                  op=Add(),
                  right=BinOp(
                     left=Constant(value=3),
                     op=Mult(),
                     right=Constant(value=4)))

BinOp(
            left=Constant(value=4),
            op=Mult(),
            right=Constant(value=2))
```

AST код вызова print:

```
Expr(
         value=Call(
            func=Name(id='print', ctx=Load()),
            args=[
               JoinedStr(
                  values=[
                     FormattedValue(
                        value=Name(id='course', ctx=Load()),
                        conversion=-1),
                     Constant(value=': '),
                     FormattedValue(
                        value=Name(id='hours', ctx=Load()),
                        conversion=-1),
                     Constant(value=' часов')])]))
```

DIS загрузка констант:

```
            LOAD_CONST               1 ('Python')
            LOAD_CONST               2 (': ')
            LOAD_CONST               3 (' часов')
            LOAD_CONST               4 (None)
```

DIS вызов функции:

```
5           LOAD_NAME                0 (print)
              PUSH_NULL
              LOAD_NAME                1 (course)
              FORMAT_SIMPLE
              LOAD_CONST               2 (': ')
              LOAD_NAME                2 (hours)
              FORMAT_SIMPLE
              LOAD_CONST               3 (' часов')
              BUILD_STRING             4
              CALL                     1
              POP_TOP
              LOAD_CONST               4 (None)
              RETURN_VALUE
```

Контрольные вопросы

Почему байт-код CPython нельзя считать машинным кодом процессора?

Ответ: байт-код CPython не предназначен для выполнения физическим процессором напрямую. Данный код выполняется виртуальной машиной Python (PVM). Данный код является более высокоуровневым, чем машинный код.

ЗАДАНИЕ 2

Код программы:

```
a = 1000
b = a
c = int("1000")

print("Типы:", type(a), type(b), type(c))
print("Идентификаторы:", id(a), id(b), id(c))
print("a == b:", a == b) # Прогноз: True
print("a is b:", a is b) # Прогноз: False
print("a == c:", a == c) # Прогноз: True
print("a is c:", a is c) # Прогноз: False
print("c is None:", c is None)



first = "python"
second = "py" + "thon"

print(f"{id(first)} {id(second)}")
print(first == second)
print(first is second)
```

Прогнозы выполнения

1. a == b будет True, так как b = a.
2. a is b будет False, так как это 2 разные переменные в памяти.
3. a == c будет True, так как c = 1000 после преобразования.
4. a is c будет False, так как это 2 разные переменные в памяти.

Результат

3 из 4 прогнозов оказались верными.

Ошибка была в том, что a is c возвращает True, а не False, так как инструкция b = a создаёт ссылку на тот же объект, на который указывает a.

Схема связей между именами и объектами

- Имена a, b → Объект 1 (значение = 1000).
- Имя c → Объект 2 (значение = 1000).

Разница между равенством значений и идентичностью объектов

Операция сравнения значения (==) проверяет, равны ли значения между двумя переменными. Данная операция не проверяет, указывают ли имена переменных на один объект. Операция is, наоборот, проверяет, указывают ли имена переменных на один объект.

Объяснение результата

Дан следующий код:

```
first = "python"
second = "py" + "thon"

print(first == second)
print(first is second)
```

Результат запуска этой программы будет следующим:

```
True
True
```

Возникает следующий вопрос: почему второй print выводит True, если две переменные должны иметь разные адреса в памяти? Это происходит из-за интернирования строк (string interning). Для оптимизации работы с памятью и повышения скорости сравнения CPython автоматически сохраняет короткие строки, похожие на идентификаторы (состоящие из букв, цифр и знаков подчёркивания), в специальном внутреннем кэше. Это и есть интернирование.

Когда применять ==, а когда is

Применять == нужно, когда проверяется равенство данных двух объектов. Применять is нужно, когда надо проверить, указывают ли переменные на один и тот же конкретный объект в памяти.

Контрольные вопросы

Создаёт ли инструкция b = a копию объекта?

Ответ: нет, данная инструкция создаёт ссылку на объект, на который ссылается переменная a.

Что возвращают type() и id()?

Ответ: type() возвращает тип данных переменной, а id() возвращает адрес в памяти, к которому ссылается переменная.

Почему None принято сравнивать с помощью is, а числа и строки — с помощью ==?

Ответ: числа и строки принято сравнивать через ==, так как данная операция сравнивает данные двух переменных, а не их адреса в памяти. None принято проверять через операцию is, так как None является одним единственным объектом в памяти, поэтому проще проверить, что переменная указывает на память, где находится объект None.

ЗАДАНИЕ 3

Разница между операциями / и //

Операция // делит 2 числа и выводит только целую часть полученного результата. Если 2 числа являлись целочисленными (int), то и тип результата будет целочисленным. Операция / делит два числа и возвращает результат без округления. Тип результата всегда float, даже если 2 числа являются целочисленными и результат является целым.

Большие числа

Целые числа в Python имеют неограниченную разрядность, и их нельзя переполнить, как в других языках с фиксированной разрядностью (C, C++, Java). Python динамически выделяет память под число, и если число выходит за рамки 64 бит, то число будет работать через алгоритм больших чисел.

Представление чисел с плавающей точкой

Python, как и большинство языков программирования, использует стандарт IEEE 754 для представления дробных чисел в двоичной системе счисления. 1 бит числа используется для знака, 11 бит — для порядка (экспоненты) числа и 52 бита — для мантиссы (дробной части). Из-за данного стандарта могут возникать неточности при вычислениях дробных значений (самый известный пример 0.1 + 0.2). Это происходит из-за того, что данное число невозможно точно представить в двоичной системе. Из-за этого приходится округлять число, чтобы оно было максимально приближено к оригиналу. Для безопасного сравнения дробных чисел следует использовать функцию math.isclose(), которая не учитывает отклонения, которые могут появиться из-за вышеупомянутой проблемы.

Преобразование типов

bool("False") выдаёт True, потому что строка не является пустой. Преобразование строки в bool выдаст False, если строка будет пустой. В

остальных случаях преобразование выдаст True. С числами работает почти так же: 0 преобразуется в False, а остальные любые числа — в True.

Контрольные вопросы

Почему результат обычного деления двух объектов int имеет тип float?

Ответ: результат деления двух целых чисел не всегда даёт целое число, поэтому обычное деление всегда даёт дробное число для упрощения. Если нужно получить в результате целое число, используйте деление с округлением до целого (//).

В чём отличие преобразования int("42") от int(42.9)?

Ответ: в первом случае мы преобразуем строку, содержащую только цифры, в число, а во втором случае мы получаем целую часть дробного числа 42.9.

Какие значения из эксперимента D являются false?

Вот код эксперимента:

```
print(int("42"))
print(float("3.14"))
print(str(2026))
print(bool(0))
print(bool(-1))
print(bool(""))
print(bool("False"))
print(complex(2, -3))
```

Результат False выдадут значения bool(0) и bool(""). Я упоминал выше о том, почему это так.

ЗАДАНИЕ 4

Ошибка присваивания

Вот код программы:

```
symbol = "Я"
symbol[0] = "д"
```

Код ошибки в строке 2:

```
TypeError: 'str' object does not support item assignment
```

Причина ошибки: в Python строки (str) являются неизменяемыми (immutable) типами данных. Вы можете обратиться к любому символу строки по индексу для чтения, но изменять её содержимое «на месте» нельзя.

Контрольные вопросы

Почему len(symbol) и len(symbol.encode("utf-8")) могут различаться?

Ответ: первая функция считает количество символов (логических юникодных кодовых точек), а вторая — количество байт, которое эти символы занимают в памяти при кодировании в UTF-8.

Что означают границы и шаг в записи text[start:stop:step]?

Ответ: start означает, с какого элемента начинается срез (по умолчанию 0). stop означает, с какого элемента срез заканчивается (по умолчанию последний элемент). step означает, с каким интервалом брать элементы (по умолчанию 1).

Почему строка считается неизменяемым объектом, хотя методы вроде .upper() возвращают изменённый текст?

Ответ: когда вы вызываете метод вроде .upper(), язык программирования создаёт абсолютно новую строку в новом месте памяти и возвращает её вам, оставляя оригинал нетронутым.

ЗАДАНИЕ 5

Код программы:

```
explorer_name = input("Имя исследователя: ")
experiment_name = input("Имя эксперимента: ")
runs_count = int(input("Количество выполненных запусков: "))
run_time = float(input("Длительность одного запуска в секундах:
"))
real_value = float(input("Действительная часть комплексного
числа: "))
imag_value = float(input("Мнимая часть комплексного числа: "))

total_time_sec = run_time * runs_count
total_time_min = total_time_sec / 60
complex_num = complex(real_value, imag_value)
module_square = real_value ** 2 + imag_value ** 2
has_runs = bool(runs_count)

print("========================================")
print(f"ЭКСПЕРИМЕНТ: {experiment_name}")
print(f"Исследователь: {explorer_name}")
print(f"Запуски: {runs_count}")
print(f"Общее время: {total_time_sec} с ({total_time_min:.2f}
мин)")
print(f"Коэффициент: {complex_num}")
print(f"Квадрат модуля: {module_square:.2f}")
print(f"Есть выполненные запуски: {has_runs}")
print("========================================")

print("Тип имени исследователя:", type(explorer_name))
print("Тип названия эксперимента:", type(experiment_name))
print("Тип количества запусков:", type(runs_count))
print("Тип времени запуска:", type(run_time))
print("Тип реальной части числа:", type(real_value))
print("Тип мнимой части комплексного числа:", type(imag_value))
```

Типы переменных

1) explorer_name — String

2) experiment_name — String

3) runs_count — int

4) run_time — float

5) real_value — float

6) imag_value — float

7) total_time_sec — int

8) total_time_min — float

9) complex_num — complex

10) module_square — float

11) has_runs — bool

Разбиение арифметической строки на литералы

Дана строка:

```
module_square = real_value ** 2 + imag_value ** 2
```

Подробный разбор строки:

1) module_square — имя переменной;

2) = — оператор;

3) real_value — имя переменной;

4) ** — оператор;

5) 2 — литерал;

6) + — оператор;

7) imag_value — имя переменной;

8) ** — оператор;

9) 2 — литерал.

Путь программы: от кода до выполнения

1) Исходный код — обычный текстовый файл с расширением .py и кодировкой UTF-8.

2) Токенизация — специальный модуль разбивает программу на токены (ключевые слова, идентификаторы, литералы, скобки и т. п.).

3) Синтаксический анализ — данный модуль проверяет, правильно ли расставлены токены, и если всё правильно, то модуль строит синтаксическое дерево.

4) Абстрактное синтаксическое дерево — на основе синтаксического дерева (CST) компилятор создаёт абстрактное дерево (AST), из которого можно скомпилировать байткод.

5) Байт-код — из абстрактного синтаксического дерева создаётся байт-код для виртуальной машины Python.

6) Выполнение кода виртуальной машиной (PVM) — машина построчно идёт по байт-коду и создаёт уже процессорные команды на C.

Почему результат input() необходимо явно преобразовывать перед арифметическими вычислениями

Потому что input() всегда возвращает строку, даже если пользователь ввёл число, поэтому для чисел нужно явно преобразовывать результат input.

Контрольные вопросы

Что в этой программе изменится на уровне объектов и связывания имен после повторного присваивания нового значения уже существующему имени??

Ответ: В памяти создастся новый объект со значением, имя отвяжется от старого объекта со значением и привяжется к новому объекту. Если на старый объект не указывает хотя бы одно имя, то объект удаляется сборщиком мусора.

КРАТКИЙ ВЫВОД

Какие 3 особенности Python меня удивили больше всего:

1) Интернирование строк — Python для оптимизации сохраняет в памяти единственную копию одинаковой строки и просто даёт ссылки объектам.

2) Кэширование малых чисел — для оптимизации Python создаёт массив чисел в памяти от -5 до 256 и все переменные, у которых эти значения, указывают на единственный объект в памяти. Если число больше, то в памяти оно создаётся отдельно.

3) Арифметика больших чисел — в отличие от многих других языков программирования с фиксированной разрядностью чисел, Python поддерживает большие числа и операции над ними.